

Big Data Management – P2

Second Follow-Up Deliverable

Cymatics: Sound-Driven Pattern Generation and Analysis Pipeline

Arman Bazarchi

Brisa Fernanda Cisneros Cervantes

Contents

1	Architecture	1
2	Trusted Zone Implementation	2
2.1	Design	2
2.2	Storage Organization	2
2.3	Current Pipeline	2
3	Exploitation Zone Implementation	2
3.1	Design	2
3.2	Generated Features	2
3.3	Storage Organization	3
3.4	Pending Work	3
4	Data Consumption Implementation	3
5	Orchestration and Containerization	4
6	Architecture Design	4

Here we present the current implementation status of the architecture proposed during the first follow-up submission.

Our focus of this report is on system integration, communication between zones, orchestration of the pipeline and storage organization, as well as on quality of transformations and analytical outputs, and we mainly describe the current state of each zone and identify the components that remain under development for the final submission.

1 Architecture

We continue with the architecture proposed in the first deliverable, following the object-storage-centric approach described as Option 1 in the project guidelines.

The system is organized around MinIO as the central storage layer, while Apache Spark provides distributed batch processing capabilities and Delta Lake serves as the analytical storage format for structured metadata, orchestration via airflow and Kafka integrated only in hot-path ingestion streaming.

Table 1: Implemented architecture layers

Layer	Storage	Metadata Storage	Processing Technology
Landing Zone	MinIO	CSV / Parquet / Delta	Python + Kafka
Trusted Zone	MinIO	CSV / Parquet / Delta	Spark + Python
Exploitation Zone	MinIO	Delta Lake	Spark + Python
Consumption Layer	Delta Lake	KPI Queries	Pandas / Python

The implemented architecture preserves the separation between data stages by having independent storage locations (buckets) for each zone in MinIO:

Structured metadata is synchronized to Delta Lake tables after end of each processing stage(one table per zone), providing a query-friendly representation for downstream analytical tasks of our data at any stage.

Semi-structured Kafka events are first archived in MinIO within landing-zone and later merged with our structured metadata during Trusted Zone processing, since Kafka messages in our pipeline contain a small and relatively stable schema (e.g., timestamps, device information, ...), making them straightforward to flatten and incorporate into a tabular representation rather than storing them separately which introduces more complexity. Unstructured assets, including audio recordings, cymatics images and generated videos, remain stored in MinIO.

2 Trusted Zone Implementation

2.1 Design

In Trusted Zone, we transform metadata rows into clean and consistent records that can be used by next stage to derive audio features or generate embeddings, this includes removing duplicated rows using UUID identifiers, validating schema and type enforcement, and merging Kafka event metadata into the structured schema. We also generate cymatics images and videos from audio recordings.

Later we plan to also include unifying all audio files into a single WAV format at this stage to ensure consistency.

2.2 Storage Organization

Trusted Zone assets including Audio recordings, Cymatics images and videos, Structured metadata (CSV, Parquet and Delta) are stored inside the MinIO bucket. Metadata is organized under a dedicated `metadata/` directory while media assets are grouped according to their dominant frequency and UUID.

2.3 Current Pipeline

The current workflow consists of two processing stages:

1. Spark batch processing for metadata cleaning.
2. Python processing for cymatics generation and metadata persistence.

Execution can be triggered manually through the orchestrator or automatically through Airflow DAGs which is set for every 2 weeks (the DAG file for landing-zone cold path is set for every 1 week) to have at least 2 batches of raw ingested data for processing.

3 Exploitation Zone Implementation

3.1 Design

In Exploitation Zone, at the moment we provide an analysis-ready representation of trusted data. In our current implementation all analytical attributes (ingested + extracted features) are stored in a single wide table. This reduces query complexity and aligns with analytical workloads described in the course material. provided processing for this zone can be triggered via the orchestrator manually or automatically via provided Airflow DAG file which is set for every 15 days, since DAG file for trusted zone is set for 14 days.

3.2 Generated Features

The implemented enrichment process extracts spectral descriptors from audio recordings. Features that we derive via Spark include Spectral centroid, Spectral bandwidth, Spectral rolloff, Spectral flatness, Signal energy, Spectral entropy, Zero-crossing rate, Loudness.

Additionally, some of our needed features require processing that are not suitable to be processed via Spark due to per row processing, therefore, we use Python-based processing to compute MFCC coefficients and Harmonic energy ratio.

3.3 Storage Organization

Our tabular data is stored in the `exploitation-zone` bucket in MinIO and synchronized to Delta Lake exploitation table, which combines Trusted metadata, Cymatics quality metrics, Audio feature vectors.

3.4 Pending Work

We are currently extending this stage to generate embeddings from our observations and integrate Milvus for embedding storage to allow similarity-search functionality and sound classifications, additionally we might improve KPIs query optimization for Data Consumption stage by providing materialized KPI tables, we might also add more pre-defined queries for more analytical uses.

4 Data Consumption Implementation

Current Consumption Tasks: As we have planed to include multiple tasks in the data consumption layer, currently we have implemented one of those tasks which lists some predefined queries as KPIs that mostly would be used for integrating with our features available in exploitation zone, defined queries are available in provided notebook to discover and in orchestration file for retrieving results and analysis at any time, this would cover the audio-visual and frequency correlation analysis usage.

- **Dominant frequency per category** Implemented via `kpi_01`, which identifies the most frequent peak frequency within each sound category.
- **Best cymatics candidates(most detailed patterns)** Implemented via `kpi_02`, which ranks recordings based on a combined cymatics quality score derived from symmetry and pattern stability.
- **Distribution across frequency bands (low / mid / high)** Implemented via `kpi_03`, grouping recordings into predefined frequency ranges and summarizing their distribution.
- **Spectrally similar categories** Implemented via `kpi_04`, which compares category-level mean peak frequencies to identify spectrally close classes.
- **Processing cost across ingestion sources** Implemented via `kpi_05`, measuring average trusted-zone processing time for warm, hot, and cold ingestion paths.
- **Spectral complexity and tonal brightness/darkness** Implemented via:
 - `kpi_06` (spectral entropy)
 - `kpi_07` (spectral centroid analysis)

Data is accessed directly from exploitation-zone Delta tables. In the final submission, this stage would additionally include, Similarity search using embeddings, and sound/pattern classification via training a model.

5 Orchestration and Containerization

The architecture extends the orchestration framework developed during Part 1.

Containerized services currently include, MinIO, Kafka, zookeeper, Apache Spark and Airflow(both handled via custom images available in project repo), PostgreSQL, SonarQube. All components are orchestrated using Docker Compose and can be fully initialized through a single `docker compose up -d` command.

Pipeline execution is available through both:

- A command-line orchestrator (`orchestrate.py`)
- Scheduled Airflow DAGs(for each stage)

6 Architecture Design

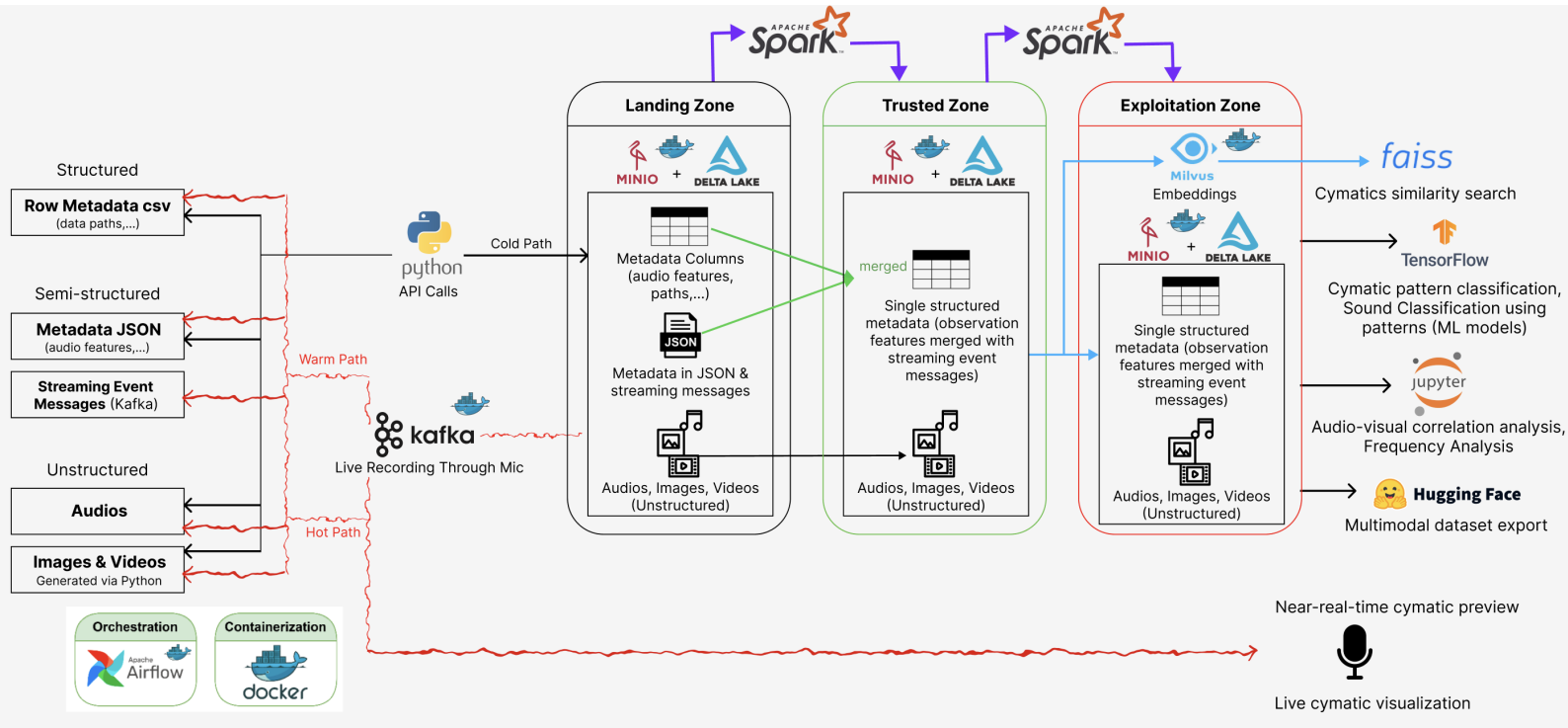


Figure 1: Architecture Design